Business Flow :-

Scenario: Citizen Reports a Digital Arrest Scam

Rahul receives a suspicious phone call from someone claiming to be a Cyber Crime officer. The caller threatens legal action unless Rahul immediately transfers money.

Instead of making the payment, Rahul opens the **Citizen Mobile Application (React)** and decides to report the incident.

Step 1 – Complaint Submission

Rahul fills in the complaint form by entering details such as:

- Phone number
- Description of the incident
- Screenshot of the message
- Call recording

After reviewing the information, he clicks **Submit Complaint**.

The application packages all the information into a request and securely sends it over the internet to the platform.

The request first reaches the **API Gateway (Kong)**, which acts as the main entry point of the platform. It verifies Rahul's identity, checks whether the request is valid, and forwards it to the appropriate backend service.

Step 2 – Case Registration

The request is forwarded to the **Case Management Service (FastAPI)**.

This service is responsible for handling every fraud complaint submitted by citizens.

It verifies that all mandatory information has been provided, checks whether a similar complaint already exists, and creates a brand-new investigation case.

A unique Case ID is generated so the complaint can be tracked throughout its lifecycle.

The complaint details are then stored in **PostgreSQL**, which serves as the primary database for transactional business data.

At this point, Rahul immediately receives an acknowledgement that his complaint has been successfully registered.

Step 3 – Evidence Processing

While the complaint is being registered, the uploaded files are forwarded to the **Evidence Service**.

Large files such as screenshots, voice recordings, videos, and PDF documents are stored securely in **Object Storage (MinIO/AWS S3)**, which is designed specifically for handling large files efficiently.

Instead of storing the files inside the database, only their metadata—such as filename, upload time, file location, and associated Case ID—is recorded.

This allows the platform to retrieve evidence quickly whenever investigators need it.

Step 4 – Preparing AI Analysis

Once both the complaint and its supporting evidence have been successfully registered, the platform forwards the required information to the **Inference Orchestrator Service**.

The orchestrator does not perform any AI processing itself.

Instead, it gathers all the relevant information, prepares it in the required format, and sends it to the independent **AI Platform** for analysis.

This separation allows the AI models to evolve independently without affecting the rest of the platform.

Step 5 – AI Analysis

The **AI Platform** receives the request and begins analyzing the submitted evidence.

Depending on the complaint, the AI models may perform tasks such as:

- Detecting scam patterns
- Identifying suspicious phone numbers
- Detecting deepfake voices
- Calculating fraud risk
- Discovering relationships with previous fraud cases

Once the analysis is complete, the AI Platform sends its prediction back to the distributed platform.

Step 6 – Updating the Investigation

The prediction is received by the Inference Orchestrator, which records the AI result and forwards it to the **Case Management Service**.

The Case Management Service updates the investigation by attaching the AI prediction, confidence score, and current risk level to Rahul's case.

If the prediction indicates a high-risk fraud, the platform can automatically prioritize the investigation.

Step 7 – Updating Other Platform Services

Once the case has been updated, the platform informs other backend services that new information is available.

The **Entity Graph Service (Neo4j)** updates the relationships between phone numbers, bank accounts, devices, UPI IDs, and other entities that may be linked to the fraud.

The **Search Service (OpenSearch)** refreshes its search indexes so investigators can immediately search for the new complaint.

The **Notification Service** prepares alerts for both Rahul and the assigned investigators.

Finally, the **Audit Service** records every important action performed during this workflow, creating a permanent history of the investigation.

Step 8 – Investigator Review

Later, a cybercrime investigator logs into the **Investigator Dashboard**.

Instead of manually collecting information from multiple systems, the dashboard presents a unified view of Rahul's case.

The investigator can immediately access:

- Complaint details
- Uploaded evidence
- AI analysis
- Fraud risk score
- Connected entities
- Investigation timeline
- Previous related cases

All this information is available in one place, allowing the investigator to quickly understand the situation.

Step 9 – Investigation Completion

After reviewing the evidence and taking the necessary actions, the investigator updates the case status.

The **Reporting Service** then generates a detailed investigation report containing the complaint history, evidence, AI findings, investigator actions, and final outcome.

The report becomes part of the official case record and can be used for further legal or administrative processes.

Architectural Flow :-

This section explains **how the request flows through the distributed platform** and how each service collaborates to process a citizen complaint. The focus is on the **architectural components and their responsibilities**, without diving into implementation details.

Step 1: Citizen Enters the Platform

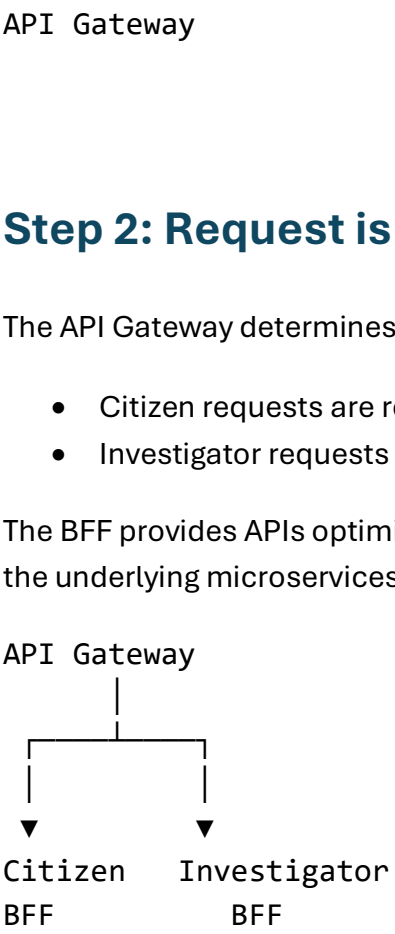
A citizen submits a fraud complaint using the mobile or web application.

The request always enters the platform through the **API Gateway**, which acts as the single entry point for all client requests.

Responsibilities

- Authenticate the user
- Validate incoming requests
- Apply rate limiting
- Route the request to the appropriate backend

```
Citizen
   |
   ▼
```



Step 2: Request is Routed to the Appropriate BFF

The API Gateway determines the type of client making the request.

- Citizen requests are routed to the **Citizen BFF**
- Investigator requests are routed to the **Investigator BFF**

The BFF provides APIs optimized for a specific client without exposing the complexity of the underlying microservices.

Step 3: Case Registration

The Citizen BFF forwards the complaint to the **Case Management Service**.

The Case Management Service is the **owner of all investigation cases**.

Responsibilities

- Validate complaint data
- Create investigation case
- Manage case lifecycle
- Assign investigators
- Maintain case timeline

Step 4: Evidence Processing

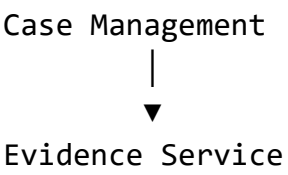
The uploaded files are forwarded to the **Evidence Service**.

The Case Management Service does not manage files.

The Evidence Service exclusively owns all digital evidence.

Responsibilities

- Store uploaded evidence
- Maintain evidence metadata
- Verify evidence integrity
- Link evidence to investigation cases



Step 5: AI Analysis Request

Once sufficient information is available, the Case Management Service requests analysis through the **Inference Orchestrator**.

The orchestrator acts as a bridge between the distributed platform and the independent AI Platform.

It does **not perform AI inference** itself.

Responsibilities

- Collect required inputs
- Prepare AI requests
- Send requests to the AI Platform
- Receive predictions

```
Case Service
      |
      ▼
Inference Orchestrator
      |
      ▼
AI Platform
```

Step 6: AI Prediction

The AI Platform analyzes the complaint and returns:

- Fraud Risk Score
- Confidence Score
- Supporting Predictions

The Inference Orchestrator receives the prediction and updates the Case Management Service.

```
AI Platform
      |
      ▼
Inference Orchestrator
      |
      ▼
Case Service
```

Step 7: Platform-wide Updates

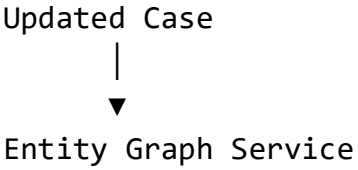
Once the case is updated, other platform services are informed.

Rather than calling every service directly, the platform distributes the update so that each service performs its own responsibility independently.

Entity Graph Service

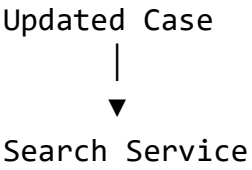
Maintains relationships between:

- Phone Numbers
- Bank Accounts
- Devices
- Wallets
- Citizens



Search Service

Updates searchable indexes so investigators can quickly search complaints, entities, and evidence.



Notification Service

Sends notifications to citizens and investigators.

```
Updated Case
      |
      ▼
Notification Service
```

Audit Service

Records every important action for compliance and traceability.

```
Updated Case
      |
      ▼
Audit Service
```

Step 8: Investigator Access

When an investigator logs into the platform, the request is routed through the **Investigator BFF**.

Unlike the Citizen BFF, it aggregates information from multiple services to provide a complete investigation dashboard.

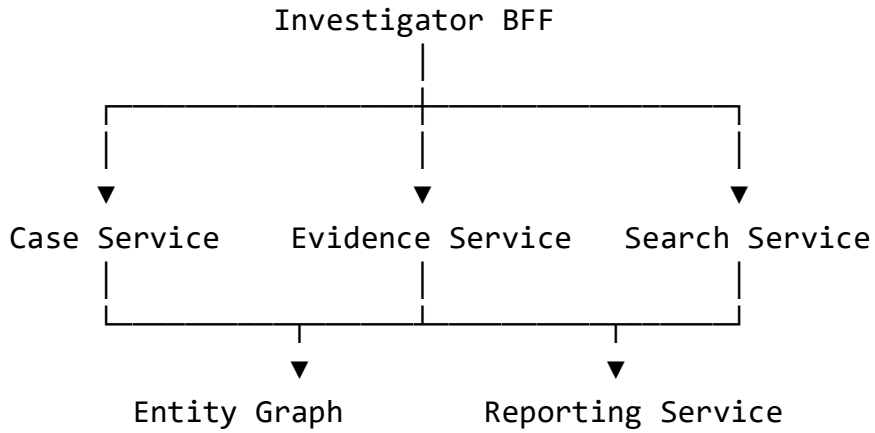
```
Investigator
      |
      ▼
API Gateway
      |
      ▼
Investigator BFF
```

Step 9: Dashboard Aggregation

The Investigator BFF gathers information from multiple backend services, including:

- Case Management Service
- Evidence Service
- Entity Graph Service
- Search Service
- Reporting Service
- Notification Service

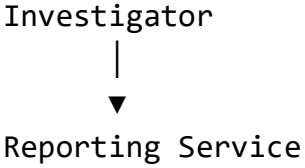
It combines the responses into a single optimized payload for the dashboard.



Step 10: Investigation Completion

After reviewing the evidence and AI prediction, the investigator updates the case status.

The **Reporting Service** generates investigation reports, evidence summaries, and court-ready documents.



```
      |
      ▼
Final Investigation Report
```

Service Responsibility Summary

Service	Primary Responsibility
API Gateway	Secure entry point and request routing
Citizen BFF	Citizen-facing APIs and response aggregation
Investigator BFF	Investigator-facing APIs and dashboard aggregation
Case Management Service	Investigation case lifecycle
Evidence Service	Digital evidence management
Inference Orchestrator	Communication with the AI Platform
AI Platform	Fraud detection and ML inference
Entity Graph Service	Fraud relationship graph
Search Service	Fast search and indexing
Notification Service	Alert delivery
Audit Service	Immutable audit trail
Reporting Service	Investigation and legal reports

Technological Flow :-

This section explains the complete journey of a request through the platform, highlighting the technology responsible at every stage.

Step 1: Citizen Submits a Complaint

The citizen interacts with the **React + TypeScript** web/mobile application.

When the user clicks **Submit Complaint**, the frontend sends a secure **REST API** request over **HTTPS**.

```
Citizen
      |
React + TypeScript
      |
HTTPS REST Request
      ▼
API Gateway
```

Why React?

- Fast and responsive UI
- Component-based development
- Easy state management
- Large ecosystem

Step 2: API Gateway

The request first reaches the **Kong API Gateway**.

The gateway is the only public entry point into the platform.

It performs:

- JWT Authentication
- Request Validation
- Rate Limiting
- API Routing
- Correlation ID Generation
- TLS Termination

- API Versioning

After successful validation, the request is forwarded to the appropriate Backend For Frontend (BFF).

```
Internet
      |
Load Balancer
      |
Kong API Gateway
      |
Citizen BFF
```

Why Kong?

- High-performance API Gateway
- Built-in authentication plugins
- Rate limiting
- Request routing
- Easy scalability

Step 3: Citizen Backend For Frontend (BFF)

The **Citizen BFF** is implemented using **FastAPI**.

It acts as a client-specific backend.

Responsibilities:

- Accept citizen requests
- Aggregate backend responses
- Transform data for frontend
- Hide internal microservices

```
Kong
      |
Citizen BFF (FastAPI)
```

```
      |
Case Service
```

Why FastAPI?

- Extremely fast
- Excellent async support
- Automatic API documentation
- Native Python ecosystem

Step 4: Case Management Service

The request reaches the **Case Management Service**.

This service contains the business logic.

It:

- Validates the complaint
- Generates Case ID
- Creates investigation
- Maintains case lifecycle

The service stores transactional data in **PostgreSQL**.

```
Citizen BFF
      |
Case Service (FastAPI)
      |
PostgreSQL
```

Why PostgreSQL?

- ACID transactions
- Strong consistency
- Foreign key support
- Excellent reliability

- Ideal for transactional systems

Step 5: Transactional Outbox

Instead of directly publishing events, the Case Service writes both:

- Case Information
- Event Information

inside the same PostgreSQL transaction.

```
Case Service
      |
PostgreSQL
 ├── Cases
 └── Outbox
```

Why Transactional Outbox?

Prevents the **dual-write problem**, ensuring database updates and events remain consistent.
Without Outbox – Let every time a case is created it is saved to the Cases database and is published in Kafka. Let's suppose Kafka is down and at that time a case is created and written in Cases database but not published in Kafka then the complaint will be missed.

With Outbox – When an outbox is introduced along with the Cases database. It will be an atomic transaction to write in both in Cases and Outbox. So even if Kafka is down, the data will be stored in the Outbox and when it is available again, the data will be published in Kafka again.

Step 6: Outbox Publisher

A background worker continuously monitors the Outbox table.

Whenever a new event appears, it publishes the event to **Apache Kafka**.

```
Outbox
      |
Outbox Publisher
      |
Kafka
```

Why Kafka?

- High throughput
- Reliable event streaming
- Loose coupling
- Independent consumers
- Horizontal scalability

Step 7: Evidence Storage

Uploaded files are sent to the **Evidence Service**.

The Evidence Service stores:

- Images
- Videos
- Audio
- PDFs

inside **MinIO** (development) or **AWS S3** (production).

Only metadata is stored in PostgreSQL.

```
Evidence Service
      |
 ├── PostgreSQL (Metadata)
 └── MinIO / AWS S3 (Files)
```

Why Object Storage?

Object storage is optimized for:

- Large files
- Scalability
- Durability
- Low storage cost

Step 8: AI Request Preparation

The **Inference Orchestrator** receives the required information.

It:

- Collects evidence
- Collects metadata
- Builds AI payload
- Sends request to AI Platform

```
Case Service
      |
Inference Orchestrator
      |
AI Platform
```

Why an Orchestrator?

Separates business logic from machine learning logic.

The backend remains independent of AI implementation.

Step 9: Internal AI Queue

Before reaching the AI Platform, requests to enter an internal durable queue.

```
Inference Orchestrator
      |
Internal Queue
```

```
      |
AI Platform
```

Why a Queue?

- Smooths traffic spikes
- Prevents AI overload
- Supports retries
- Improves reliability

Step 10: AI Platform

The AI Platform is an independent Python-based platform responsible only for ML inference.

It performs:

- Scam Detection
- Deepfake Detection
- Counterfeit Detection
- Fraud Risk Prediction
- Network Analysis

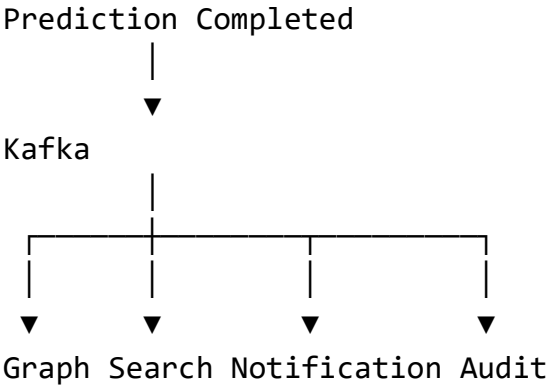
The prediction is returned to the Inference Orchestrator.

```
AI Platform
      |
Prediction
      ▼
Inference Orchestrator
```

Step 11: Event Distribution

The orchestrator stores predictions and publishes an event.

Kafka distributes the event to multiple independent services.



Why Kafka?

Each service reacts independently without tightly coupling to the others.

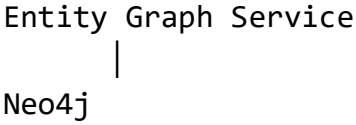
Step 12: Entity Graph Service

The **Entity Graph Service** updates fraud relationships.

Relationships include:

- Citizens
- Phone Numbers
- Bank Accounts
- UPI IDs
- Devices
- Wallets
- IP Addresses

These relationships are stored in **Neo4j**.



Why Neo4j?

Optimized for relationship traversal and fraud network analysis.

Step 13: Search Service

The Search Service updates searchable indexes using **OpenSearch**.

Instead of querying multiple databases, investigators search a centralized index.

```
Search Service
      |
OpenSearch
```

Why OpenSearch?

- Full-text search
- Fast filtering
- Near real-time indexing
- Scalable search

Step 14: Notification Service

The Notification Service sends:

- SMS
- Email
- Push Notifications
- Dashboard Updates

Real-time dashboard updates use **Server-Sent Events (SSE)** or **WebSockets**.

```
Notification Service
      |
SMS
```

```
Email
Push
WebSocket
```

Step 15: Audit Service

Every important action is stored by the Audit Service.

Audit logs are immutable.

```
Audit Service
      |
Append-only Audit Database
```

Why?

Provides traceability and legal compliance.

Step 16: Investigator Dashboard

The investigator logs in through the **Investigator BFF**.

The BFF gathers information from multiple backend services using **gRPC**.

```
Investigator
      |
Gateway
      |
Investigator BFF
      |
gRPC
      ▼
Multiple Microservices
```

Why gRPC?

- Faster than REST
- Binary Protocol Buffers
- Lower latency
- Efficient service-to-service communication

Step 17: Dashboard Response

The Investigator BFF aggregates responses from:

- Case Service
- Evidence Service
- Entity Graph Service
- Search Service
- Reporting Service

It returns a single optimized response.

Step 18: Reporting

The Reporting Service generates:

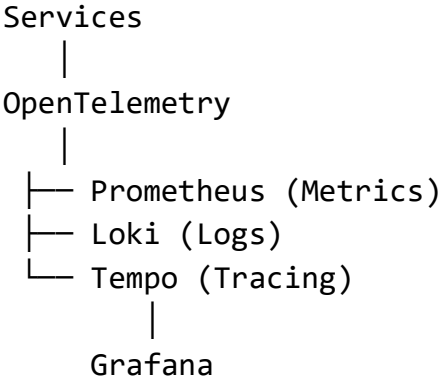
- Investigation Reports
- Evidence Summaries
- Court Packages
- PDF Reports

Generated reports are stored in Object Storage.

Step 19: Observability

Every service automatically exports:

- Metrics
- Logs
- Traces



Why?

Allows engineers to monitor system health, debug issues, and trace requests across microservices.

Step 20: Deployment

All services are packaged as **Docker containers**.

The containers are deployed on **Kubernetes**.

Kubernetes manages:

- Service discovery
- Auto Scaling
- Self Healing
- Rolling Updates
- Load Balancing

```
Docker Containers
        |
Kubernetes Cluster
        |
Pods
```

Why Kubernetes?

Provides a resilient, scalable, cloud-native deployment platform.

Complete Technology Stack Summary

Layer	Technology	Purpose
Frontend	React + TypeScript	User Interface
API Gateway	Kong	Authentication, Routing, Rate Limiting
BFF	FastAPI	Client-specific APIs
Business Services	FastAPI	Core business logic
ORM	SQLAlchemy	Database interaction
Relational Database	PostgreSQL	Transactional data
Object Storage	MinIO / AWS S3	Evidence storage
Event Streaming	Apache Kafka	Asynchronous communication
AI Queue	Internal Durable Queue	Buffer AI requests
AI Platform	FastAPI	Machine Learning inference
Graph Database	Neo4j	Fraud relationship graph
Search Engine	OpenSearch	Fast search
Internal Communication	gRPC	Service-to-service communication
Cache	Redis	Sessions, caching, temporary data
Monitoring	Prometheus + Grafana	Metrics and dashboards
Logging	Loki	Centralized logs

Tracing	OpenTelemetry + Tempo	Distributed tracing
Containerization	Docker	Package services
Orchestration	Kubernetes	Deployment, scaling, self-healing